

The Effects of Team Backlog Dependencies on Agile Multiteam Systems: A Graph Theoretical Approach

Alexander Scheerer
SAP SE &
University Mannheim
scheerer@uni-mannheim.de

Saskia Bick
SAP SE &
University Mannheim
bick@uni-mannheim.de

Tobias Hildenbrand
SAP SE
tobias.hildenbrand@sap.com

Armin Heinzl
University Mannheim
heinzl@uni-mannheim.de

Abstract

In agile software development, the prioritization of backlog items is the mission-critical responsibility of the product owners in order to maximize the customer value created by development teams. However, in the reality of large-scale development, the degree of freedom for such a prioritization is substantially restricted by various types of interdependencies between backlog items. In this work, we show, using a graph theoretical approach, the relation between the degree of freedom for prioritization and the occurrence of dependencies. To the best of our knowledge, the breadth and depth of such consequences has never been modeled or investigated up until now. Based on our results, we derive implications for real-world large-scale software development in agile environments.

1. Introduction

In agile software development, the prioritization of requirements in the form of *backlog items* is the mission-critical responsibility of *product owners* (“the sole person responsible for managing the backlog” [41:5]), both on a small scale, starting with a single team, as well as on a larger scale, i.e. in a multiteam system setting [25,38]. Single or multiteam, a backlog “is an ordered list of everything that might be needed [...] and is the single source of requirements for any changes to be made” [41:12]. The degree of freedom (DoF), i.e. the relative amount of choice, compared to a backlog without dependencies, to freely prioritize team backlog items, is an essential instrument regarding not only team empowerment but also customer centricity in order to maximize the customer value created by development teams [38]. In reality, however, large-scale agile software development, and accordingly the DoF while prioritizing team backlog items, is substantially restricted by various types of dependencies between items.

Ideally, empowered development teams in a multiteam system setting should face a low degree of inter-team dependencies and a higher degree of intra-team

dependencies to allow for effective multiteam system coordination and efficient intra-team work [45]. Yet, in real-world examples of large-scale agile development environments, product owners are often complaining about being responsible for prioritizing requirements and creating value for customers, while the stage is already set due to given dependencies. As described in the Scaled Agile Framework (SAFe) by Leffingwell [26], cross-team features in the shared program backlog are split into single-team backlog items, i.e. user stories, with tentative allocations to individual sprints, which limits the product owners and their teams in their “free choice” of backlog item prioritization.

According to the agile development paradigm, maximizing customer value and thus ensuring economic product viability for the company is a key principle [38]. Nevertheless, products are rarely built from scratch and – as a timely delivery of new functionality for standard solutions is often based on existing platforms, components and frameworks – legacy code may impose several restrictions for the development of new code [17].

These constraints on the product owner’s possibility to influence the order of the backlog are not only a highly relevant practical problem [4,24,26], but previous research has also called for more studies on the underlying fundamental concepts of agile software development [1,2,16]. Dependencies on an inter-team level are an under-researched fundamental concept which still lacks scientific depth [cf. 4,8,14]. The research question we seek to answer is: *What are the impacts of dependencies in the team backlog and how do these effects emerge on a multiteam level?*

As a first step towards solving this problem, our research aims at providing insights into consequences of dependencies in team backlogs in large-scale agile software development and suggesting two main approaches how to deal with existing dependencies. In this work, we show – using a simulative graph theory approach – the relation between level of dependencies and the degree of freedom for prioritization of backlog items. Based on our results, we derive implications for real-world large-scale software development in agile

environment in that we highlight the severity of dependencies in general and for multiteam systems in particular.

Our paper is structured as follows. In the foundations chapter, we depict current findings on agile software development, dependencies and coordination theory in order to provide the necessary knowledge for the subsequent modeling approach chapter. Here, assumptions and modeling items as well as the mathematical procedure are explained. The results are critically reflected thereafter in the discussion section. We conclude our work with future research implications.

2. Foundations

Starting with a brief overview on agile software development, we illustrate the environment surrounding our simulative study. After introducing large-scale agile and multiteam systems, a concept stemming from organizational psychology research, we provide a short overview on dependencies. Subsequently, we introduce the conceptual fundamentals of coordination.

2.1. Agile Software Development

In 2001, several prominent advocates of lightweight development methods gathered to create the Agile Manifesto [6]. They proposed four values which constitute the essence of agile development methods: individuals and interactions, working software, customer collaboration and responding to change. These values illustrate the considerable mind shift in agile software development. They emphasize cross-functional and empowered teams with time-boxed development iterations and continuous management of requirements.

Agile project management frameworks, such as the Scrum approach [40], and large-scale agile development approaches, e.g. the SAFe [26], which builds upon the former, both suggest work management through ordered backlogs. On a team level, such a backlog contains all items one team will directly work on. In agile software development, these backlog items are often represented as *user stories*, which describe a feature from the perspective of a certain user role including the value it provides in his/her context and gives the development team enough freedom to work out the details in implementation. The team can pull one or more of these items from the team backlog into their *sprint backlog*, i.e. for a timeframe of two to four weeks. After discussing the user stories and estimating the efforts, the team then commits these items for the sprint. A unique prioritization is recommended [41].

This prioritization is carried out by the product owner, who serves as the representative of the custom-

ers and thus should be able to order the backlog according to customer priority and value. The prioritization of requirements is one of the key functions in assuring customer focus, mitigating risk, fast value delivery and successful steering of a project. However, this job is strongly affected by dependencies between individual backlog items or user stories in particular.

Regarding this field of research, a considerable number of studies have been conducted concerning agility on a team level, some of which are still asking for more rigorous research and detailed insights as opposed to reports of mere anecdotal evidence [1,2,16]. Concerning large-scale agile development research, insights are even more limited [15].

2.2. Large-Scale Agile Development and Multiteam Systems

Large-scale agile development very closely builds upon the agile software development paradigm described before. In fact, it extends the latter by implying the collaboration of multiple teams which all make use of agile methodologies. On the practical side, several approaches, e.g. Scrum-of-Scrums [23]; Dynamic Systems Development Method [12] or Disciplined Agile Delivery [3], have been developed to provide a guiding structure for a multiteam setup. In our consideration of the research question, we will utilize the Scaled Agile Framework as an example [26]. Here, a program backlog contains potential product features which are further broken down into smaller user stories to nourish the team backlogs. User stories of one larger feature can be worked on by several different development teams. The individual product owner of a development team is henceforth provided with user stories coming from the higher level program backlog. Accordingly, he/she has to deal with pre-determined, and partly unknown, dependencies in the team backlog and is therefore limited in the DoF to prioritize the backlog [26].

Coming from organizational theory, another view on large-scale development systems emerged. This type of organizational setup, where several teams have to work together in order to complete a release of a development product, has been described within the organizational psychology domain as a multiteam system (MTS). The collective goal of this system can be broken down into a goal hierarchy and constitutes a key characteristic of any MTS. The goal hierarchy marks the boundary of an MTS in that all teams within the system share at least a distal goal while the individual teams pursue their more proximal goals individually. This structure of goals leads to teams displaying input, process and outcome interdependence with at least one other team [29,39].

Whereas a certain level of research in the organizational psychology community already exists to provide us with insights on multiteam systems and resulting dependencies in general, research on dependencies in the large-scale agile development setting in particular is still rather scarce [4,5,15,24]. Learning from extant research, we aim to deepen the knowledge in the agile software development context as well as transfer our insights to real-world large-scale development settings.

2.3. Requirements Prioritization and Ordering

The term *priority* has historically been used in ambiguous ways, implying different concepts to different stakeholders. The definition of priority according to two English language dictionaries exemplifies this statement (see Table 1). While (2), (3) and (5) deal with a certain order in time or scheduling, (1) and (4) describe *relative* importance. These two concepts can also be seen as the basis for the prevalent views on prioritization. A widespread conception for requirements prioritization is to attach a priority to individual activities based on value, risk or cost (among many others). An example of this approach is the MoSCoW method [9], which describes four categories of deliverables: must, should, could and won't. It becomes apparent that this approach describes the relative importance between individual requirements. However, Firesmith [18:37] argues, that "requirements prioritization is determining the implementation order of requirements and that prioritization by importance is merely one means to that end". As such it has been suggested to use the term *order* to be more precise, which of the two meanings of prioritization is intended. Orders' sole meaning, as can be seen in Table 1, is the arrangement of activities in time.

This focus can also be seen in the decision of the authors of the Scrum Guide [41] to change the wording from *priority* or *prioritized backlog* to *ordered backlog*. This emphasizes the goal of creating a list which has a *unique* order in which to process backlog items [13].

For this paper, we make use of the following conceptualization: Prioritization means to assign a relative value/importance to one particular activity. Ordering refers to a unique sequence of several activities. It is the job of the product owner to first assign priorities (and he is limited in doing so in a completely free manner by pre-determined dependencies) and then to order these prioritized backlog items in a unique way.

Table 1. Definitions of key terms.

Priority	(1) a preferential rating (2) the quality or state of being prior (3) something given or meriting attention before competing alternatives [31] (4) A thing that is regarded as more important than another (5) The right to take precedence or to proceed before others [34]
Order	The arrangement or sequence of objects or of events in time. [32] The arrangement or disposition of people or things in relation to each other according to a particular sequence, pattern, or method. [35]
Dependency	A situation wherein an activity cannot proceed until another activity is complete. [44:6]
dependent	(1) Unable to do without (2) Contingent on or determined by [36]

2.4. Dependencies and Coordination

We view the previously sketched research problem through the lens of coordination theory, which defines coordination as the management of dependencies [28].

In the literature, the terms *dependency* and *interdependency* are used interchangeably and refer both to a unilateral relationship, i.e. *a* is dependent on *b* but not the other way round. Accompanying and resulting from coordination theory, there are different taxonomic approaches to characterize dependencies, the most important of which we present subsequently. The widely used term *task* is to be understood as *work* or *activity* in this context. A traditional view was proposed by Thompson [45], who classified task dependencies into *pooled*, *sequential* and *reciprocal* types. In this typology, interdependence is conceptualized as a need for activity alignment. Based on this taxonomy, Van de Ven et al. [46] further specified the participating entities as workers within an organizational unit who are dependent upon each other in order to fulfil their individual tasks. Furthermore, the authors introduced a fourth type of interdependence: *team* interdependence. This state is present when work is done jointly and simultaneously by employees of one organizational unit at the same time. Yet another dependency taxonomy provided by Crowston [14] focuses on the relationship between *tasks* and *resources* which can

arise in organizational settings and proposes different combinations of these two entities, namely shared resources, producer-consumer and common output. Strode et al. [44] view dependencies as the antecedent of potential or actual constraints which might impact the success of a specific activity. The authors name three types of dependencies: *knowledge*, *task* and *resource* dependencies, which each comprise multiple sub-categories.

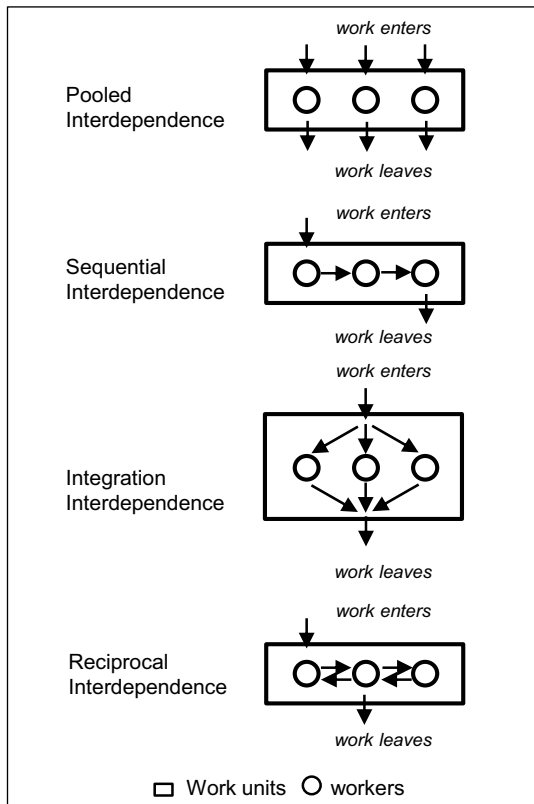


Figure 1. Categorization of Task Interdependencies [22,45]

In our research, we view a dependency as “a situation wherein an activity cannot proceed until another activity is complete” [44:6], i.e. backlog item A must be implemented first, as it depends on backlog item B (see also Table 1).

For this study, in the context of MTS in large-scale agile development, we make use of the relatively new concept of *integration interdependence* [22], as this addresses the integration challenges of several teams working together on one software product. Kumar et al. describe four characterizing aspects of integration interdependence. First, the overall work activity, in our case the program backlog feature, is subdivided into several sub-activities for multiple parties to work on separately. Second, not the individual activities themselves, but the whole, aggregated sum of the parts is of

value. Third, to reintegrate the parallel working streams, an integration process is needed. Fourth, all members of the system collaborating in a setting of integration interdependence have to be aware of the status, potential issues and the progress of their co-teams as their own output is heavily dependent on the work of the others. Figure 1 shows the three classical interdependence types by Thompson [45] as well as the recent concept of integration interdependence by Kumar et al. [22].

As already described in chapter 2.2, these circumstances influence the degree of freedom to prioritize the team backlog items for the product owner. To be more precise, out of these integration interdependencies, other order dependencies, i.e. unilateral dependencies that express constraints and a specific directionality, arise in the team backlog and have to be considered when prioritizing a specific work sequence. In order to manage these dependencies, coordination mechanisms are needed to actively manage the backlog. Yet, before exploring resolution and prevention strategies, it is of importance to first closely investigate consequences of dependencies in agile backlogs. To the best of our knowledge, the breadth and depth of such consequences has never been modeled or investigated up until now.

In the next chapter, we introduce our modeling approach and present guiding assumptions and used parameters.

3. Modeling Approach

The approach taken within this research, to model a team backlog in a large-scale agile setting, is a simulative graph theoretical one. We initially chose this path as the problem presented itself to be of a multi-dimensional nature. Further understanding was needed in order to analytically tackle this complex issue. For our modeling approach, we consider the backlog of one team, as the multiteam system consists of several individual teams working together. In order to fully comprehend dependencies in the multiteam system, one has to first understand the dependencies emerging at the individual team which originate from the multiteam setting. We view this as the first step in understanding the complex dependencies in the full MTS.

The following paragraphs explain how we model the different entities relevant to this research and their representation in graph theoretical terms. Details on the calculations conclude this chapter.

3.1. Team Backlogs, Backlog Items and Dependencies

The *team backlog* is modeled as a directed acyclic graph (DAG), i.e. a graph with no cyclical arrangement of vertices. We model each *backlog item* as vertices within the DAG. Further, we view any kind of relationship between backlog items which necessitates that one be finished before a different one as a dependency between these two items. The edges between vertices in our DAG represent the *order dependencies* between individual backlog items.

In summary, we model each team backlog with v items and e dependencies as a directed acyclic graph with unlabeled nodes and v vertices as well as e edges. We represent the backlog as a graph with unlabeled nodes, as the backlog items are all of the same type, in our case a user story, and the structure of the graph is essential, not the naming of nodes. Hence, we only consider one graph of each isomorphism class in our approach. The number of DAGs with unlabeled nodes is shown in Table 2.

We limit our simulative examination to backlogs with a maximum amount of ten items due to reasons of super-exponential growth of this integer sequence.

Table 2. Number of acyclic digraphs with n unlabeled nodes [42]

n	$A(n)$
0	1
1	1
2	2
3	6
4	31
5	302
6	5 984
7	243 668
8	20 286 025
9	3 424 938 010
10	1 165 948 612 902

3.2. Calculating the Degree of Freedom

Degree of Freedom. We view the relative amount of choice, compared to a backlog without dependencies, to freely prioritize team backlog items as the *degree of freedom*. This is defined as the ratio of valid permutations P_v in a backlog with dependencies to all possible permutations P_a , reflecting the portion of independent prioritization available such that $DoF = \frac{P_v}{P_a}$.

Implicit or Transitive Dependencies. By specifying dependencies within the backlog it is possible to have implicitly generated more dependencies as the initial specification would suggest. This situation can be seen in Figure 2. To account for transitive dependencies, we calculated the transitive closure of each DAG, which adds these implicitly specified dependencies to the graph representation of each backlog.

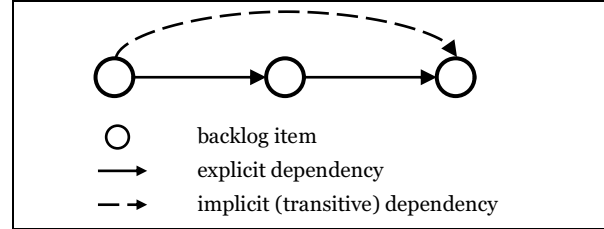


Figure 2. Implicit (Transitive) Dependencies

Valid Permutations. A permutation which respects the dependencies such that for each directed edge $v1v2$ from vertex $v1$ to vertex $v2$, $v1$ comes before $v2$ is defined as a topological order. Thus, the amount of topological orders for each transitive closure is the amount of valid permutations for the specified DAG. At least one, but usually several orders can be calculated, which represent the ways in which the backlog can be processed while fulfilling the dependencies specified.

In summary, we establish the DoF by calculating the ratio of the number of topological orderings of each transitive closure of the specified DAG with edges and the number of topological orderings of the DAG without edges (see Figure 3).

$$DoF = \frac{\text{\# of topological orderings of transitive closure of DAG with dependencies}}{\text{\# of topological orderings of DAG without dependencies}}$$

Figure 3. Definition of the Degree of Freedom

We have generated our data set with the help of Sage Mathematics Software [33,43]. Hereby, we calculated the topological orderings for all unlabeled DAGs with nodes 1 to 8 [30] and created 18 million random DAGs with 9 vertices as well as 20 million random DAGs with 10 vertices. The previously mentioned super-exponential growth of increasingly larger DAGs makes the smaller subset of DAGs with 9 and 10 vertices necessary. In total, we calculated the DoF of 58 658 547 representations of a backlog with dependencies.

4. Results and Discussion

The degrees of freedom, including a trend line for increasing amounts of dependencies, can be seen for 2 to 10 item backlogs in Figure 5.

In answering the question of dependency impact on the team backlog, our main findings are (1) the DoF exhibits very strong decay, (2) this decay is independent of backlog size and (3) the structure of dependencies seems to be essential as only ranges of DoF could be identified for most combinations of parameters.

One dependency already decreases the DoF by 50% and this number drops below 20% as soon as we pass four dependencies (see Figure 5). The data shows strikingly, an increased amount of dependencies dramatically lowers the DoF. This confirms our initial expectation in chapter 2.2, that predetermined dependencies have a significant impact on the prioritization ability of the team product owner. In fact, the product owner is drastically limited in freely assigning priorities to the backlog items and subsequently ordering them accordingly. One dependency between two backlog items, for example, will already reduce his freedom of choice in how to prioritize the backlog by 50%.

The independence of DoF from backlog size initially seems counterintuitive, but can be explained by our definition of DoF, which normalizes the number of topological orderings of the backlog with dependencies by the topological orderings of the same backlog without dependencies.

What quickly becomes obvious is that for the tuple *backlog size – dependency amount*, only a range of possible DoF can be specified. This also holds true for the combination of other parameters which we calculated (Connected Components, Density, average Distance, Diameter, Radius, max In/Out-Degree, min In/Out-Degree, In/Out-Degree Sequence). In Figure 4 we provide an excerpt of the DoF for an 8 item backlog based on the number of bound items and the amount of dependencies which clearly shows the previously mentioned ranges. These ranges suggest that the structure of the order dependencies is essential in understanding the exact consequences of dependencies in team backlogs.

With regard to our initially stated research question, we see that the previously discussed effects emerge on the multiteam system level in a way that the greater the interdependence between work activities, the higher the coordination effort resulting in a higher chance of failure and delay [22]. Due to the system of integration interdependencies, delays on a team level have direct effects on the higher multiteam system level. If one team cannot deliver in time what it committed to for the sprint, other teams and thus the whole multiteam system might experience delays in a cascading manner.

Influenced by the level of existing dependencies in a team backlog, the resulting degree of freedom impacts the team's planning complexity and thus its coordination effort. This impact is particularly severe as a certain level of remaining flexibility in the product owner's freedom to prioritize items in the team backlog is needed to be able to react to unforeseen events as one of the key values of agile development [21]. This in turn affects the overall multiteam system predictability and delivery reliability [22].

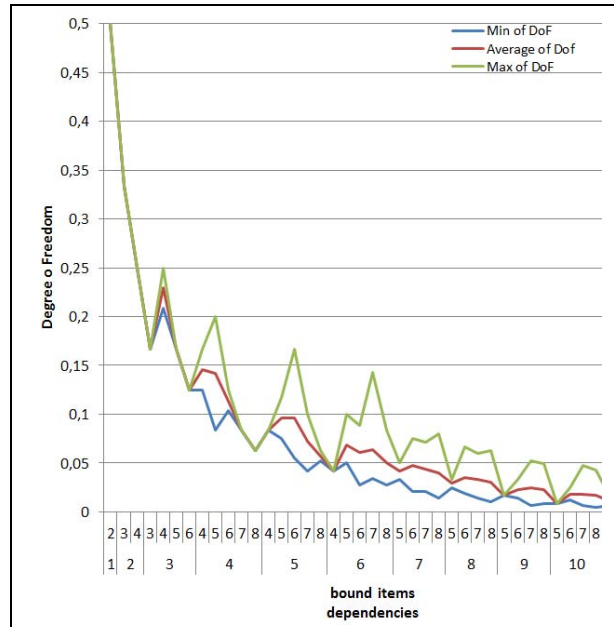


Figure 4. Excerpt showing minimum, average and maximum DoF for a 8 item backlog

5. Implications and Future Research

We have managed to gain first insights into the consequences of dependencies in team and program backlogs in large-scale agile software development systems. Even few dependencies in the team backlog dramatically reduce the DoF for prioritization of the team product owner. This impact is independent of backlog size and has a severe impact on the DoF.

The surrounding MTS setting has an impact on its embedded teams regarding the dependencies in team backlogs, as the team product owner has to consider pre-determined dependencies coming from assigned user stories in his/her team backlog. In the other direction, the effects of dependencies in the team backlog and the according prioritization in turn strongly influence the MTS level through the integrative nature of dependencies on the program backlog level. It is therefore essential to consider ways in which to deal with these types of dependencies

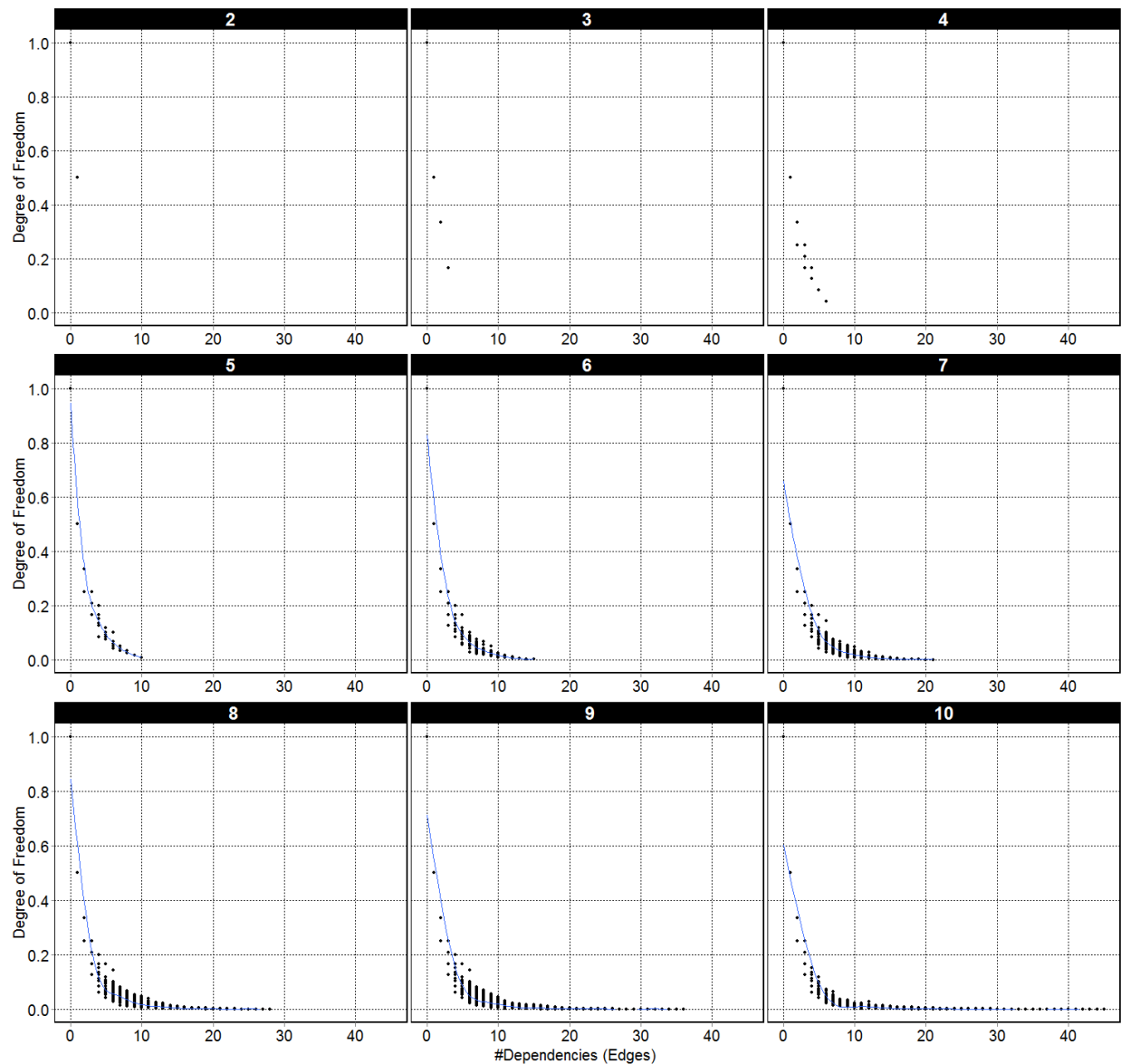


Figure 5. Degrees of freedom in 2 - 10 item backlogs

The findings so far suggest two approaches how to deal with dependencies in large-scale settings. The first one refers to the management of dependencies of which a few heuristics and methods have emerged lately. Splitting requirements (“user stories”) in smaller chunks [25,38] as well as approaches which attempt to generate an overview of the connections between requirements (e.g. user story mapping [37] or walking skeleton [11]) are mechanisms which make requirement dependencies transparent early on.

Creativity techniques in upfront software development stages, such as Design Thinking [10], in combination with user story mapping are said to help creating a more solid product vision and more relevant re-

quirements to start with [19]. This in turn helps to minimize functional dependencies and make them transparent across development teams and business units. The described techniques used in the context of larger features can also help to split and resolve user stories in the course of a project. There are also approaches to capture dependencies collaboratively in the course of the project while providing visual analysis and editing functions in real time [20].

Once requirements have been defined, many companies use backlog management and tracking tools to further improve transparency and coordinate work across multiple teams.

The other approach advocates the avoidance of dependencies altogether. Especially strategies concerning the composition of multiteam systems and teams in particular are worth mentioning. The current discussion on feature vs. component teams is a step in this direction (cf. [8]). The architecture of the software also plays a strong role in dependency avoidance. Here, modularization strategies from open source development, i.e. plug-in architectures, could help in this regard (cf. [27]).

Besides these highly relevant practical implications, our study also provides theoretical contributions. First, based on a literature review on agile development and dependencies, our study required carving out the subtleties of the existing interdependency taxonomies. In particular, applying our findings to the multiteam level contributes to the body of knowledge on the concept of integration interdependence. Doing so, we have been able to deepen the knowledge on interdependencies on different analytical levels. Furthermore, we sought to answer the persistent call for more rigorous studies on agility in software development by examining the under-researched, yet tremendously important phenomenon of dependencies in agile backlogs. While the amount and quality of studies on agility in large-scale development settings constantly increases, the effects of dependencies at the core of agile work teams, namely their ordered list of requirements, have mostly been disregarded.

Future studies could build upon the presented findings and pursue an analytical approach to mitigate the limitations of the previously discussed interval results. In our research, this approach has so far proved unsuccessful as the parameters to determine a distinct degree of freedom remained elusive. At the same time, specific dependency patterns, i.e. the structure of order dependencies in simulative calculations could be of immense interest.

Speaking of limitations, the simulative character of our study could be regarded as such. Furthermore, we only looked at one type of dependency at this point. Reciprocal dependencies are just one potential other type of dependencies which might be of interest in the future. In addition, our focus on the team backlog, although justified, implies certain limitations for the generalizability of our findings.

Future research should therefore concentrate on the following main areas: on the one hand, the need for deeper insights on inter-team dependencies, in large-scale development requires more empirical studies in a real-life setting. With respect to this organizational structure, the effect of different types of coordination and dependencies as well as mitigation strategies and how they affect each other and the development system seem to be a promising line of inquiry.

In this light, the actual impact of certain methods for resolving and avoiding dependencies, e.g. splitting requirements and user story mapping need to be researched more in depth. So far, only practitioner reports are available which state positive effects of these methods over numerous projects.

6. References

- [1] Abrahamsson, P., Conboy, K., and Wang, X. 'Lots done, more to do': the current state of agile systems development research. *European Journal of Information Systems* 18, 4 (2009), 281–284.
- [2] Ågerfalk, P.J., Fitzgerald, B., and Slaughter, S. Flexible and distributed information systems development: State of the art and research challenges. *Information systems research* 20, 3 (2009), 317–328.
- [3] Ambler, S.W. and Lines, M. *Disciplined Agile Delivery: A Practitioner's Guide to Agile Software Delivery in the Enterprise*. Pearson Education, 2012.
- [4] Bannerman, P.L., Hossain, E., and Jeffery, R. Scrum Practice Mitigation of Global Software Development Coordination Challenges: A Distinctive Advantage? *System Science (HICSS)*, 2012 45th Hawaii International Conference on, (2012), 5309–5318.
- [5] Barlow, J.B., Giboney, J.S., Keith, M.J., Wilson, D.W., and Schuetzler, R.M. Overview and Guidance on Agile Development in Large Organizations. *Communications of the Association for Information Systems* 29, 2 (2011), 25–44.
- [6] Beck, K., Beedle, M., Bennekum, A. van, et al. Manifesto for Agile Software Development. *Agile Alliance*, 2001.
- [7] Begel, A., Nagappan, N., Poile, C., and Layman, L. Coordination in Large-Scale Software Teams. *Proceedings of the 2009 ICSE Workshop on Cooperative and Human Aspects on Software Engineering (CHASE '09)*, (2009), 1–7.
- [8] Blau, B. and Hildenbrand, T. Product Line Engineering in Large-Scale Lean and Agile Software Product Development Environments - Towards a Hybrid Approach to Decentral Control and Managed Reuse. *Availability, Reliability and Security (ARES)*, 2011 Sixth International Conference on, 2011, 404–408.
- [9] Brennan, K. *A Guide to the Business Analysis Body of Knowledge*. International Institute of Business Analysis, 2009.
- [10] Brown, T. *Change by Design*. HarperCollins, 2009.
- [11] Cockburn, A. *Crystal clear: A Human-Powered Methodology for Small Teams*. Addison-Wesley Professional, Amsterdam, 2004.
- [12] Consortium, D. *DSDM Atern Handbook*. 2014. <http://www.dsdm.org/dig-deeper/book/dsdm-atern-handbook>.

- [13] Coplien, J. It's Ordered -- Not Prioritized! 2011. <https://www.scrumalliance.org/community/articles/2011/august/it's-ordered—not-prioritized!>
- [14] Crowston, K. *A Taxonomy of Organizational Dependencies and Coordination Mechanisms*. 1994.
- [15] Dingsøyr, T. and Moe, N.B. Research Challenges in Large-Scale Agile Software Development. *ACM SIGSOFT Software Engineering Notes*, (2013), 38–39.
- [16] Dybå, T. and Dingsøyr, T. Empirical studies of agile software development: A systematic review. *Information and Software Technology* 50, 9-10 (2008), 833–859.
- [17] Feathers, M. *Working Effectively with Legacy Code*. Prentice Hall, 2007.
- [18] Firesmith, D. Prioritizing Requirements. *Journal of Object Technology* 3, 8 (2004), 35–47.
- [19] Hildenbrand, T. and Meyer, J. Intertwining Lean and Design Thinking: Software Product Development from Empathy to Shipment. In *Software for People*. Springer, 2012, 217–237.
- [20] Hildenbrand, T. *Improving Traceability in Distributed Collaborative Software Development: A Design Science Approach*. 2008.
- [21] Kude, T., Bick, S., Schmidt, C., and Heinzl, A. Adaptation Patterns in Agile Information Systems Development Teams. *22nd European Conference on Information Systems (ECIS)*, (2014), 1–15.
- [22] Kumar, K., van Fenema, P.C., and von Glinow, M.A. Offshoring and the global distribution of work: Implications for task interdependence theory and practice. *J Int Bus Stud* 40, 4 (2008), 642–667.
- [23] Larman, C. and Vodde, B. *Practices for Scaling Lean and Agile Development: Large, Multisite, and Offshore Product Development with Large-Scale Scrum*. Addison-Wesley Professional, Upper Saddle River, N.J, 2010.
- [24] Lee, E.C. Forming to performing: Transitioning large-scale project into agile. *Agile, 2008. AGILE'08. Conference*, (2008), 106–111.
- [25] Leffingwell, D. *Agile Software Requirements: Lean Requirements Practices for Teams, Programs, and the Enterprise*. Addison-Wesley Professional, 2011.
- [26] Leffingwell, D. Scaled Agile Framework. 2014.
- [27] MacCormack, A., Rusnak, J., and Baldwin, C.Y. Exploring the Structure of Complex Software Designs: An Empirical Study of Open Source and Proprietary Code. *Management Science* 52, 7 (2006), 1015–1030.
- [28] Malone, T.W. and Crowston, K. The interdisciplinary study of coordination. *ACM Computing Surveys* 26, 1 (1994), 87–119.
- [29] Mathieu, J.E., Marks, M.A., and Zaccaro, S.J. Multiteam systems. In N. Anderson, D.S. Ones, H.K. Sinangil and C. Viswesvaran, eds., *Handbook of Industrial, Work and Organizational Psychology Volume 2 Organizational Psychology*. Sage Publications Ltd, London, 2001, 289–313.
- [30] McKay, B. (Research S. of C.S.N.U.). Combinatorial Data. <http://cs.anu.edu.au/people/bdm/data/digraphs.html>.
- [31] Merriam-Webster.com. priority. 2014. <http://www.merriam-webster.com/dictionary/priority>.
- [32] Merriam-Webster.com. order. 2014. <http://www.merriam-webster.com/dictionary/order>.
- [33] Oliphant, T.E. Python for Scientific Computing. *Computing in Science & Engineering* 9, 3 (2007), 10–20.
- [34] oxforddictionaries.com. priority. 2014. http://www.oxforddictionaries.com/definition/american_english/priority.
- [35] oxforddictionaries.com. order. 2014. <http://www.oxforddictionaries.com/definition/english/order>.
- [36] oxforddictionaries.com. dependent. 2014. <http://www.oxforddictionaries.com/definition/english/dependent>.
- [37] Patton, J. User Story Mapping. 2008. <http://guide.agilealliance.org/guide/storymap.html>.
- [38] Reinertsen, D.G. *The Principles of Product Development Flow*. Celeritas Publishing, 2009.
- [39] Scheerer, A., Hildenbrand, T., and Kude, T. Coordination in Large-Scale Agile Software Development: A Multiteam Systems Perspective. *47th Hawaii International Conference on System Sciences (HICSS)*, (2014), 4780–4788.
- [40] Schwaber, K. and Beedle, M. *Agile Software Development with Scrum*. Prentice Hall, 2001.
- [41] Schwaber, K. and Sutherland, J. The Scrum Guide. 2013. [https://www.scrum.org/Portals/0/Documents/Scrum Guides/2013/Scrum-Guide.pdf](https://www.scrum.org/Portals/0/Documents/Scrum%20Guides/2013/Scrum-Guide.pdf).
- [42] Sloane, N.J.A. Number of acyclic digraphs with n unlabeled nodes. <http://oeis.org/A003087>.
- [43] Stein, W.A. and others. Sage Mathematics Software (Version 5.4.1). 2012.
- [44] Strode, D.E. and Huff, S.L. A taxonomy of dependencies in agile software development. *23rd Australasian Conference on Information Systems (ACIS)*, (2012), 1–10.
- [45] Thompson, J.D. *Organizations in Action: Social Science Bases of Administrative Theory*. McGraw-Hill, 1967.
- [46] Ven, A. Van de, Delbecq, A., and Jr, R.K. Determinants of coordination modes within organizations. *American sociological review* 41, 2 (1976), 322–338.